

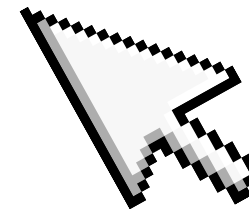
Por Gabriel Kowaleski, Jully Anne, Maria Rita e Mayra Bordin

TRABALHO 01

CONTAGEM PARALELA

DE OBJETOS EM UMA MATRIZ BINÁRIA

Start





0	1	1	0	0	0	0	0	0	0	1	0
0	0	1	1	1	1	0	0	0	1	1	0
0	0	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	1	1	0	0	0	0	0
0	1	0	0	0	0	1	0	0	1	0	0
0	1	1	0	0	0	0	0	1	1	0	0
0	0	1	1	0	0	0	0	1	0	0	0
0	0	0	1	0	0	0	1	1	0	0	0
0	0	0	0	0	1	0	0	0	0	0	1

Problema e Objetivo

- **Matriz binária Objeto** = células 1 conectadas
- **Conectividade 8**: horizontal, vertical e diagonal
- **Objetivo**: mesma contagem nas versões sequencial e paralela

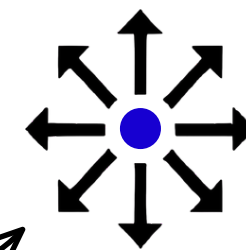


Problema Implementação Resultados



O QUE PRECISAMOS
CONTAR?

- **Matriz binária:** 0 = fundo e 1 = primeiro plano
- **Objeto** = células 1 conectadas por arestas ou cantos
- **Estratégia:**
 - Sequencia → Flood Fill
 - Paralela → Pthreads + Union-Find
- **Conectividade 8:** horizontal, vertical e diagonal



Problema Implementação Resultados

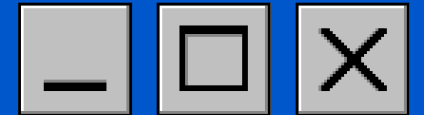


VERSÃO SEQUENCIAL



- Varre a matriz
- [1 não visitado] → novo objeto
- Flood Fill nos 8 vizinhos
- Pilha explícita: versão iterativa
- Serve como referência de correção





Problema Implementação Resultados

```
int contar_objetos_sequencial(int *matriz, int linhas, int colunas) {
    int *visitado, *pilha;
    int objetos, i, j, indice, total_celulas;

    total_celulas = linhas * colunas;

    visitado = (int *)calloc(total_celulas, sizeof(int));
    pilha = (int *)malloc(total_celulas * sizeof(int));

    if (visitado == NULL || pilha == NULL) {
        printf("Erro ao alocar memoria para a contagem sequencial.\n");
        free(visitado);
        free(pilha);
        return -1;
    }

    objetos = 0;
    for (i = 0; i < linhas; i++) {
        for (j = 0; j < colunas; j++) {
            indice = i * colunas + j;

            if (matriz[indice] == 1 && visitado[indice] == 0) {
                objetos++;
                flood_fill(matriz, visitado, linhas, colunas, i, j, pilha);
            }
        }
    }
    free(visitado);
    free(pilha);
    return objetos;
}
```



Problema Implementação Resultados



COMO EVITAMOS CONDIÇÕES DE CORRIDA?

- | | | | | |
|---|---|---|---|---|
| <ul style="list-style-type: none">• Matriz de entrada:
Somente leitura• Rótulos: Cada thread escreve apenas na sua faixa | → | <ul style="list-style-type: none">• pthread_create inicia o trabalho• pthread_join cria a barreira• union-find somente após o join | → | <ul style="list-style-type: none">• Tratamento de erros:<ul style="list-style-type: none">◦ pthread_create/pthread_join◦ malloc/calloc◦ clock_gettime◦ liberação de memória |
|---|---|---|---|---|



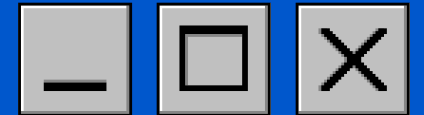
Problema **Implementação** Resultados



DECOMPOSIÇÃO PARALELA

- Faixas horizontais
- Uma região exclusiva por Thread
- Flood Fill local + rótulos
- Configuração: 2, 3 ou 4 threads
- $\text{linha_inicio} = i * \text{linhas}/\text{num_threads}$





Problema **Implementação** Resultados



SINCRONIZAÇÃO E SEGURANÇA

Por que não usamos mutex?

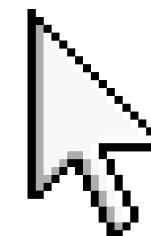
Matriz: somente leitura

Rótulos: escrita apenas na própria faixa

ThreadArgs: um elemento por thread

pthread_join encerra a fase paralela

Union-find só é usada depois do join
Retornos POSIX e alocações são verificados





ANÁLISE DAS FRONTEIRAS



**total = locais -
uniões**

UNION FIND

Cada thread rotula só a sua faixa, então o mesmo objeto pode ter 2 rótulos. Após os join, compara-se a última linha de uma faixa com a primeira da seguinte (colunas -1, 0 e +1, inclui diagonal) e une-se os rótulos equivalentes. Ex.: matriz 2, 2 threads: $2 + 3 - 1$ união = 4 objetos.

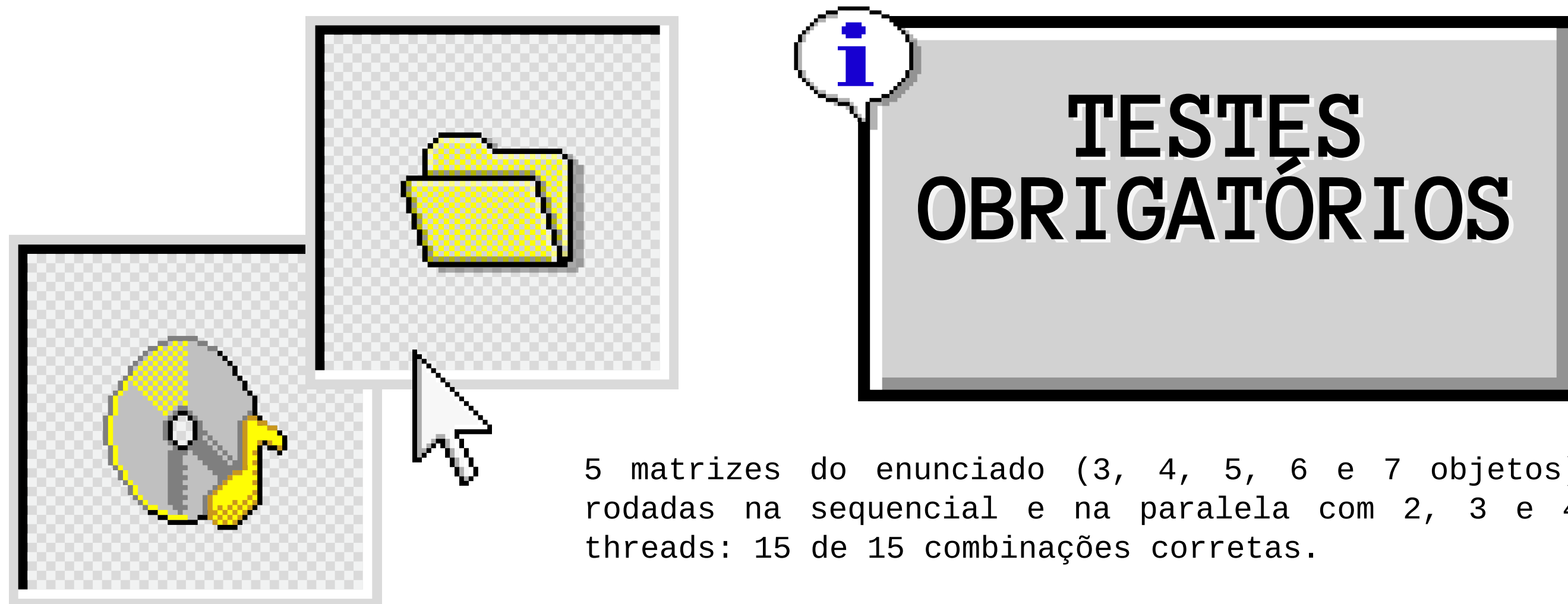


Paralelo: flood fill e rótulos de cada faixa (p threads). Sequencial: particionamento, fronteiras, Union-Find e contagem final. O pthread_join separa as duas fases.



**ONDE ESTÁ O
PARALELISMO**





**TESTES
OBRIGATÓRIOS**

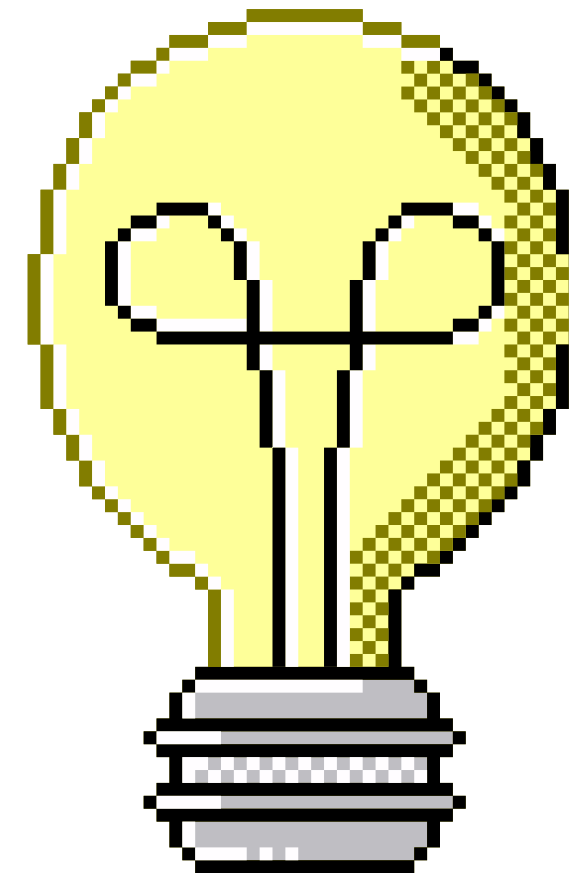
5 matrizes do enunciado (3, 4, 5, 6 e 7 objetos)
rodadas na sequencial e na paralela com 2, 3 e 4
threads: 15 de 15 combinações corretas.

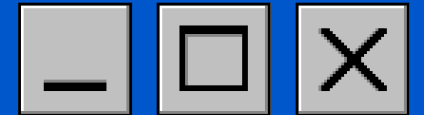




TESTES ADICIONAIS

Extras: diagonal na fronteira, objeto em 3 faixas e matriz zerada. Teste aleatório: 141.397 comparações paralela x sequencial, sem divergência. ASan, UBSan e TSan sem erros.





Base **Projeto** Final

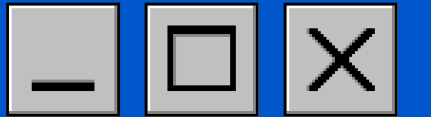
1000x1000: 17,9 ms
(seq.) e 9,5 ms com
3 threads: 1,89x.



DESEMPENHO

4000x4000: 265 ms
(seq.) e 197 ms com
3 threads: 1,35x.
Média de 5
execuções.





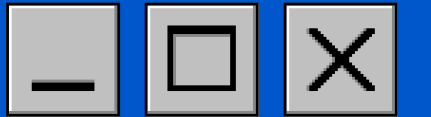
Base **Projeto** Final

3 threads foi o melhor nos 3 tamanhos; 4 threads não melhora. A eficiência cai com mais threads (0,52 / 0,45 / 0,33 na 4000x4000).
Limites: fronteiras sequenciais, memória e custo das threads.

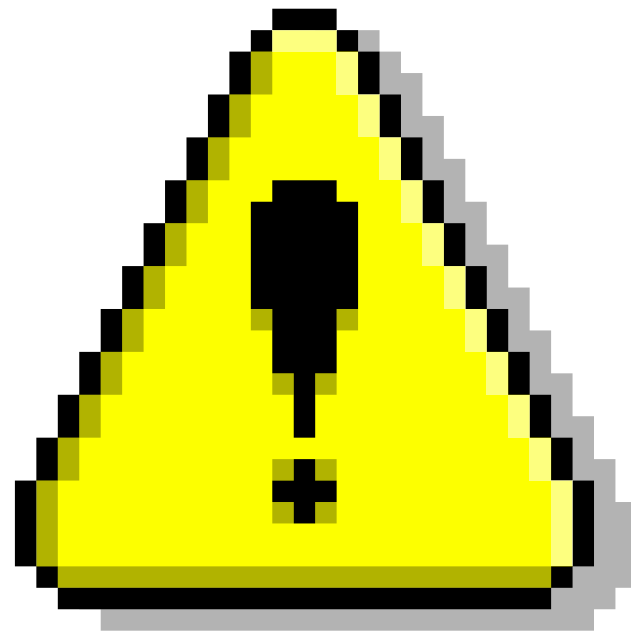


ANÁLISE





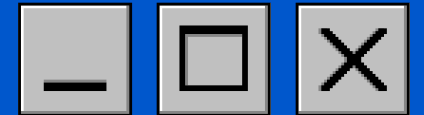
Base **Projeto** Final



DEMONSTRAÇÃO

Comandos: `make clean && make | ./sequencial | ./paralelo 3 | ./desempenho` (opcional, cerca de 1 min).





Base **Projeto** Final



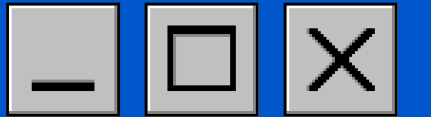
REPOSITÓRIO

Yes

Cancel

github.com/MRita011/T1-S0: código em src/, testes em tests/, medições e gráficos em results/, Makefile, README e relatório técnico.



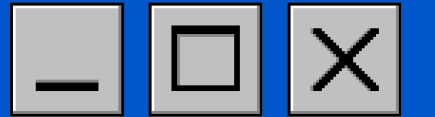


Base **Projeto** Final

Limites: testes em WSL e sem -O2,
fronteiras sequenciais, matrizes fixas no
código. Melhorias: balanceamento
dinâmico, consolidação em paralelo e
medições com -O2.

**LIMITAÇÕES E
MELHORIAS**





Base Projeto **Final**

CONCLUSION

Sequencial e paralela: mesmo resultado em todos os testes.
Melhor ganho com 3 threads: 1,35x a 1,89x. Consolidação com
Union-Find, sem mutex.

MRita011/T1-S0

Obrigado!

Turma 330

